

ResNet Implementation and Convolution Concepts Explained

1. Introducción a la ResNet

ResNet (Redes Residuales) es una arquitectura de red neuronal convolucional (CNN) innovadora que se introdujo para resolver el problema de la **degradación** en redes neuronales muy profundas. Este problema consiste en que, a medida que las redes se hacen más profundas, su rendimiento en los datos de entrenamiento puede empeorar.

La degradación ocurre porque optimizar capas adicionales se vuelve extremadamente difícil. Un modelo más superficial con un rendimiento óptimo podría ser reproducido por un modelo más profundo simplemente haciendo que las capas adicionales aprendan la función de identidad (es decir, $H(x)=x$). Sin embargo, las arquitecturas de CNN tradicionales con capas apiladas no son capaces de aprender esta función de identidad de manera efectiva, lo que lleva al empeoramiento del rendimiento.

- **Nota:** No confundir con el overfitting. Si hay presencia de overfitting, el rendimiento en los datos de entrenamiento es muy alto, pero el rendimiento en los datos de prueba (no vistos) es bajo. En la degradación, el rendimiento es peor incluso en los datos de entrenamiento, lo que indica un problema fundamental en la capacidad de la red para aprender una función de identidad.

La innovación central de ResNet es el **bloque residual** (Residual Block), que utiliza "conexiones de salto" (o conexiones de identidad) para permitir que el gradiente fluya más fácilmente a través de la red durante la retropropagación (backpropagation), mitigando el problema del gradiente desvanecido y permitiendo el entrenamiento de modelos mucho más profundos.

2. La idea central: el Aprendizaje Residual

En lugar de aprender una asignación directa de una entrada x a una salida $H(x)$, un bloque residual aprende una **función residual** $F(x) = H(x) - x$. El output final del bloque resulta ser $y = F(x) + x$.

Concepto Matemático

La operación clave resulta en la conexión que salta una o diversas capas:

$$y = \mathcal{F}(x, \{W_i\}) + x$$

- **x**: El input del bloque residual ("conexión de identidad" o "skip connection").
- **F(x, {W_i})**: El mapeo residual a ser aprendido por unas pocas capas apiladas (por ejemplo, convoluciones, normalización por lotes o batch norm, ReLU).
- **y**: Output del bloque.

Nota: $H(x)$ es la función de mapeo (o transformación) que idealmente debería realizar un bloque de capas para llevar la entrada x a la representación óptima $H(x)$ para la siguiente etapa de la red. Es el objetivo final del bloque de capas apiladas.

La adición del input de entrada permite a la red aprender una función de identidad (es decir, $y=x$) llevando los pesos de las capas apiladas (F) a cero. Esto significa que agregar más capas, en el peor de los casos, no perjudicará el rendimiento, ya que las nuevas capas pueden simplemente aprender a pasar la entrada sin cambios.

Este hecho es así porque resulta más fácil para un optimizador (como el Descenso de Gradiente) forzar los pesos W_i de las capas a cero (es decir, aprender $F(x)=0$) que forzar las capas a aprender a copiar la entrada ($H(x)=x$) en una red tradicional.

- **Redes Tradicionales:** Para lograr $H(x)=x$, las capas tendrían que encontrar un conjunto específico y complejo de pesos no nulos que mapeen la entrada exactamente a sí misma. Esto es computacionalmente difícil y rara vez se logra de manera eficiente.
- **ResNet:** Si la representación ya es óptima, el modelo solo necesita que las capas apiladas se "apaguen", aprendiendo el mapeo residual cero. Esto significa que las capas añadidas no dañarán el rendimiento (el problema de la degradación) porque tienen una vía fácil para volverse inoperantes (aprender la identidad).

3. Tipos de convoluciones implementadas

El código utiliza varios tipos de convoluciones 2D, cada una con un propósito específico. Una convolución 2D es una operación matemática donde un kernel (o filtro) se desliza sobre una imagen de entrada (o mapa de características) para producir un mapa de características de salida.

El tamaño del mapa de características de salida se calcula como:

$$\text{Output Size} = \frac{(W - K + 2P)}{S} + 1$$

- **W**: Dimensión de entrada
- **K**: Dimensión del kernel

- **P:** Padding
- **S:** Stride

3.1. Convolución 3x3 (`nn.Conv2d(..., kernel_size=3)`)

- **Explicación:** Se trata de la capa convolucional estándar empleada en las CNNs modernas.
- **Propósito:** Se utiliza para extraer características espaciales de la entrada. Un núcleo de 3×3 es el tamaño más pequeño que puede capturar nociones de centro, esquinas y bordes. Apilar múltiples convoluciones de 3×3 puede crear un campo receptivo efectivo mayor, ya que dos capas 3x3 equivalen a una de 5x5 y tres capas 3x3 equivalen a una de 7x7.
- **Implementación:** Tanto en el BasicBlock como en el BottleneckBlock, la convolución de 3×3 es la capa principal para aprender patrones espaciales.

```
# From BasicBlock
self.conv1 = nn.Conv2d(in_channels, out_channels, kernel_size=3, stride=stride, padding=1,
```

Se utiliza padding=1 con un kernel_size=3 y stride=1 para preservar las dimensiones espaciales del mapa de características de entrada.

3.2. Convolución 1x1 (`nn.Conv2d(..., kernel_size=1)`)

- **Explicación:** Componente clave del BottleneckBlock y de la capa de downsample. Es uno de los trucos de ingeniería más importantes en las arquitecturas modernas como ResNet, ya que permite manipular la profundidad (número de canales) sin afectar las dimensiones espaciales.
- **Propósitos:**
 - Reducción/Expansión de Dimensionalidad:** Este es su rol principal en el BottleneckBlock. Una convolución 1×1 puede reducir el número de canales (profundidad) de un mapa de características, haciendo que la posterior convolución de 3×3 sea computacionalmente más económica. Luego se utiliza de nuevo para expandir los canales de vuelta a una dimensión mayor. Esto crea el efecto de "cuello de botella".
 - Proyección para las Skip Connections:** Cuando la la skip connection x tiene dimensiones diferentes (ya sea el tamaño espacial debido a un stride >1 o la profundidad de canales) a la salida de la función residual $F(x)$, la convolución 1×1 proyecta x al nuevo espacio dimensional para que puedan ser sumadas.
- **Matemáticamente:** Una convolución 1×1 es esencialmente una combinación lineal de los canales en cada ubicación de píxel. Mira un solo píxel a la vez y aplica una capa totalmente conectada (fully connected layer) a través de sus canales.

```
# From BottleneckBlock (reducing channels)
self.conv1 = nn.Conv2d(in_channels, out_channels, kernel_size=1, bias=False)

# From _make_layer (for downsampling)
downsample = nn.Sequential(
    nn.Conv2d(self.in_channels, out_channels * block.expansion, kernel_size=1, stride=2,
    ...
)
```

3.3. Convolución 7x7 (nn.Conv2d(..., kernel_size=7))

- **Explicación:** La capa de convolución 7×7 es una característica de diseño tradicionalmente utilizada en las primeras capas de las CNN construidas para grandes bases de datos de imágenes (como ImageNet), donde las imágenes de entrada suelen ser de alta resolución (típicamente 224×224 píxeles).
- **Propósito:** Se utiliza solo una vez, como la primera capa en las arquitecturas ResNet diseñadas para ImageNet. Un núcleo grande de 7×7 proporciona un gran campo receptivo al inicio de la red, permitiendo capturar patrones globales o características de alto nivel (como texturas generales o formas grandes) desde el principio.
- **Implementación:** La capa convolucional se emplea cuando las imágenes son grandes y complejas como es el caso de ImageNet. Para imágenes más pequeñas (como el caso de CIFAR-10 que son de 32x32 píxeles), un kernel de 7x7 puede resultar excesivo ya que podría abarcar gran parte de la imagen, perdiendo la capacidad de extraer detalles localizados y llevando a una rápida pérdida de información espacial. Por eso, para este caso se emplea el kernel de 3x3.

```
# For ImageNet
self.conv1 = nn.Conv2d(input_channels, 64, kernel_size=7, stride=2, padding=3, bias=False)
# For CIFAR-10
self.conv1 = nn.Conv2d(input_channels, out_channels=64, kernel_size=3, stride=1, padding=1,
```

4. Detalles de la implementación

Clase BasicBlock (para ResNet-18/34)

- **Explicación:** Se trata del diseño original y más simple del bloque residual. Se utiliza en modelos ResNet con menos capas, como ResNet-18 y ResNet-34. está compuesto por dos capas convolucionales de 3×3 apiladas, con la skip connection que salta ambas capas.

- **Estructura:** 3x3 Conv -> BatchNorm -> ReLU -> 3x3 Conv -> BatchNorm .
- **Forward Pass:**
 - i. El input inicial x se guarda como la identidad. Esta es la base de la skip connection.
 - ii. El input pasa por la primera Convolución (3×3), la primera normalización (BatchNorm) y la primera activación (ReLU).
 - iii. A continuación, la salida pasa por la siguiente capa convolucional y otra normalización.
Note: *No se realiza una activación ReLU después de la segunda normalización, ya que dicha activación se introduciría antes de la suma residual, lo cual no es el diseño original ni el más efectivo.*
 - iv. El output producido por el paso de las convolucionales se adiciona con la entrada guardada correspondiente a la identidad. Si las dimensiones del output y la identidad no coinciden, la identidad debe pasar por una capa de `downsample` (esta capa típicamente usa una convolución 1×1 con stride 2 para hacer que las dimensiones espaciales y el número de canales coincidan antes de la suma).
 - v. La última activación ReLU se aplica después de la suma residual, siendo este hecho fundamental porque garantiza que la no-linealidad se aplica a la función residual completa ($F(x)+x$).

Clase BottleneckBlock (para ResNet-50/101)

- **Explicación:** Se trata de un diseño más complejo y eficiente para la construcción de redes más profundas como ResNet-50 y ResNet-101 (en estas redes seguir usando el BasicBlock sería computacionalmente de gran costo). Está compuesto por tres capas convolucionales.
- **Estructura:** 1x1 Conv (compresión) -> 3x3 Conv (features espaciales) -> 1x1 Conv (descompresión).
- **Forward Pass:**
 - i. 1º Convolución: Se realiza una reducción utilizando un filtro o kernel de 1×1 para reducir drásticamente el número de canales de entrada (compresión).
 - ii. 2º Convolución: Se realiza un procesamiento, en que se aplica un kernel 3×3 sobre el número reducido de canales anteriores, haciéndola más rápida.
 - iii. 3º Convolución: Se realiza una expansión utilizando un kernel 1×1 para aumentar el número de canales a su tamaño original para que coincida con la entrada residual (descompresión). El factor de expansión (valor de 4) determina el número final de canales de salida.
 - iv. Se adiciona la skip connection del mismo modo que en el BasicBlock .

Clase ResNet

Esta clase ensambla los bloques para formar una red completa ResNet.

- **Función `_make_layer`** : Es una función auxiliar que crea una "etapa" de la ResNet (por ejemplo, `layer1` , `layer2`). Determina si se necesita una capa de `downsample` (reducción de muestreo). Esto se produce al inicio de una nueva etapa (donde el `stride` es 2) o cuando cambia el número de canales.
También, crea el primer bloque de la etapa (el cual podría tener un `stride` de 2) y luego añade los bloques restantes.
- **Capas Iniciales**: Se distingue entre la configuración para CIFAR-10 (convolución de 3×3 , sin max pooling) y la configuración para ImageNet (convolución de 7×7 , con max pooling), basándose en los parámetros de entrada.
- **Capas Finales**: Después de las capas residuales, se utiliza `nn.AdaptiveAvgPool2d` para reducir las dimensiones espaciales de cada mapa de características a 1×1 , independientemente del tamaño de entrada. A esto le sigue una capa totalmente conectada (`nn.Linear`) estándar para la clasificación.

5. Creación de la ResNet-18, ResNet-34, ResNet-50 y ResNet-101

Los valores en la lista `layers` (`[a, b, c, d]`) definen la arquitectura profunda de cada ResNet. Específicamente, indican cuántos bloques residuales se apilan en cada una de las cuatro etapas principales de la red:

- a: Número de bloques residuales en la Etapa 1 (`layer1`).
- b: Número de bloques residuales en la Etapa 2 (`layer2`).
- c: Número de bloques residuales en la Etapa 3 (`layer3`).
- d: Número de bloques residuales en la Etapa 4 (`layer4`).

Cada valor de la ResNet implementada hace referencia al número total de capas convolucionales con pesos que son contadas y que se pueden entrenar:

a) ResNet-18

- Layers `[2,2,2,2]`
- Bloques totales = 8
- Capas en bloques (Bloques x 2) = 16
- Capas adicionales = 2 (Inicial y Final)
- Total capas = 18

b) ResNet-34

- Layers [3,4,6,3]
- Bloques totales = 16
- Capas en bloques (Bloques x 2) = 32
- Capas adicionales = 2 (Inicial y Final)
- Total capas = 34

c) ResNet-50

- Layers [3,4,6,3]
- Bloques totales = 16
- Capas en bloques (Bloques x 3) = 48
- Capas adicionales = 2 (Inicial y Final)
- Total capas = 50

d) ResNet-101

- Layers [3,4,23,3]
- Bloques totales = 33
- Capas en bloques (Bloques x 3) = 99
- Capas adicionales = 2 (Inicial y Final)
- Total capas = 101

6. Resultados obtenidos

- Parámetros y FLOPS vs. Modelo:
 - ResNet-18 y ResNet-34 (usan BasicBlock) tienen menos parámetros que ResNet-50 y ResNet-101 (usan BottleneckBlock). Esto es lógico, ya que el BottleneckBlock tiene más capas y se expande a más canales finales para lograr su eficiencia.
 - La Complejidad Computacional (FLOPS) y la Latencia (Velocidad de Inferencia) aumentan con la profundidad del modelo (ResNet-18 < ResNet-34 < ResNet-50 < ResNet-101). Más capas y más operaciones se traducen directamente en más tiempo de cómputo, como se ve claramente en las gráficas inferiores.
- Relación entre FLOPS y Latencia:
 - La gráfica de FLOPS (Complejidad Computacional) y la gráfica de Latencia (Velocidad de Inferencia) son casi idénticas en forma. Esto es muy lógico, ya que la latencia en un entorno de inferencia (como el que se está midiendo) es directamente proporcional al número de operaciones (FLOPS) que la red debe realizar.
- La Precisión no Aumenta con la Profundidad:

- ResNet-18 (el modelo más pequeño) es consistentemente el que logra la mayor precisión (~ 0.85). A medida que el modelo se hace más profundo (ResNet-34, 50, 101), la precisión cae, siendo ResNet-101 el que logra la menor precisión (~ 0.52). En este caso, más profundidad sí perjudica la precisión en el conjunto de entrenamiento/prueba.

Los motivos pueden tratarse sobre:

- Overfitting al ruido: CIFAR-10 tiene solo 60,000 imágenes de muy baja resolución (32×32). Las redes más grandes (ResNet-50/101), con su enorme capacidad (millones de parámetros), tienden a sobreajustarse dramáticamente a los pequeños detalles y al ruido de los datos de entrenamiento. La ResNet-18 puede resultar mejor para capturar las características generales sin sobreajustarse al ruido del conjunto pequeño.
- ResNet-50 y 101 utilizan el BottleneckBlock, útil para la eficiencia en imágenes grandes (224×224). En imágenes más pequeñas (32×32), el BottleneckBlock (con sus filtros 1×1 que reducen y expanden) puede comprimir y luego expandir la información esencial de forma demasiado agresiva, degradando la calidad de las características antes de que los filtros 3×3 puedan actuar eficazmente. En este caso, el BasicBlock de ResNet-18/34 es más efectivo.

 Métricas Obtenidas